



ROBOCUPJUNIOR RESCUE SIMULATION 2026

TEAM DESCRIPTION PAPER — WORLD CHAMPIONSHIP

Team Name Marko Jack	Members Lovro Zigman (17) — Team Captain Luka Kolaric (18) — Co-Captain Orsat Kralj (17) — Software Support
Country Croatia	Simulation Platform Erebus (Webots)
League Rescue Simulation	

Abstract

This Team Description Paper presents the complete design, engineering decisions, and software implementation of the UNIQA CRO Team RoboCraft autonomous robot for RoboCupJunior Rescue Simulation 2026 World Championship. The robot operates inside the Erebus simulation platform (built on Webots) at 32 ms per time step, navigating procedurally generated four-room mazes, building an accurate occupancy-grid map, and detecting victims and hazard signs on the walls.

The hardware consists of a 360-degree 512-ray LiDAR, two 64x32 pixel side-facing cameras, GPS, IMU, eight distance sensors, and a colour sensor. All software is written in Python 3 (~3 200 lines) using NumPy, OpenCV, PyTorch, and Ultralytics YOLOv8. Navigation is based on a modified A* algorithm operating on a sparse-dictionary occupancy grid whose cells each store a 5x5 matrix of typed dots. Room-to-room transition is tracked through coloured tunnel tiles read by the colour sensor.

Victim detection (H, S, U symbols and C, F, O, P hazmat signs) is handled by a YOLOv8 model trained on Erebus simulation data, protected by a six-stage false-positive filter cascade. A position-aware deduplication system prevents the same victim from being reported multiple times. Map serialisation applies nine post-processing stages including BFS phantom-wall removal and a Room 4 bounding-box fill algorithm before transmission to the supervisor. A universal exception safety net ensures the robot never crashes silently.

1. Team Overview

1.1 Team Members

Member	Role & Background
Lovro Zigman (17) — Team Captain	Lead navigation and mapping developer. Designed the Cell/Net grid architecture, A* path planner, LiDAR wall-detection pipeline, obstruction recovery, room tracking, and overall software structure. Competing in RoboCup since 2024 (Rescue Line, Rescue Maze, Soccer Lightweight).
Luka Kolaric (18) — Co-Captain	AI and computer-vision developer. Built the YOLO training pipeline, the six-stage false-positive filter cascade, victim position estimation, and GPU-accelerated inference. Competing in RoboCup since 2019 across Rescue Line, Rescue Maze, Soccer Lightweight, CoSpace Rescue, and other disciplines.
Orsat Kralj (17) — Software Support	Simulation testing, sensor calibration, debugging support, and visual presentation design. Contributes to performance optimisation and documentation.

1.2 Project Timeline

Phase	Period	Key Deliverables
Research & rules study	Sep–Nov 2025	Complete understanding of Erebus API, scoring rules, sensor constraints
Architecture design	Dec 2025	Cell/Net class design, YOLO training plan, sensor fusion strategy finalised
Core development	Jan–Feb 2026	Working navigation, map building, YOLO integration, end sequence
Croatia qualifier prep	Mar 2026	Integration testing, false-positive tuning, rule compliance verification
Post-qualifier optimisation	Apr 2026	Room 4 fill algorithm, BFS phantom filter, wall-inference improvements
World championship final	May–Jun 2026	Universal safety net, LoP handler hardening, complete documentation

2. Robot Design

2.1 Sensor Suite

Each sensor was selected to serve a specific, non-overlapping role. No single sensor is trusted exclusively; every safety-critical measurement (walls, holes, obstructions) is cross-validated.

Sensor	Specification	Primary Role	Implementation Detail
LiDAR	512 rays, 360°, multi-layer; layer index 2 selected	Wall mapping, corner fill, neighbour propagation	getLidarValue() averages 3 adjacent rays; std-dev split for edge noise. ckWallLidar() uses 20 named distance ranges and a 5-ray majority vote.
Distance sensors (x8)	8 sensors; danger threshold 0.04 m; 3 named sensors (ds0, ds1, dsf) + depth_sensor	Obstruction detection, immediate safety zone	Any sensor reading < 0.04 m sets obstruction_found = True and triggers reverse-to-last-node recovery.
Camera Left / Right	64×32 px each; side-facing; FoV used for victim angle estimation	Victim & hazard detection	Frames processed every step when not turning. A pre-filter rejects frames that are >90% dark or cyan wall before calling YOLO.
GPS	Continuous X/Z position in metres	Grid localisation	getCoords() computes $\text{round}(\text{coord} * 100 / 6)$ to map GPS metres to integer grid units. Non-rounded version used for dynamic turn targeting.
IMU	Full roll/pitch/yaw	Heading tracking, turn control	getAngle() converts radians to 0–360° (0 = north, clockwise). getRotation() divides by 45 and rounds to an integer 0–7 for octagonal heading index.
Colour sensor	1-pixel BGRA floor reading	Tile classification (checkpoints, swamps, tunnels)	getColor() unpacks struct BGRA, converts to HSV. checkColor() validates against per-tile HSV range dictionaries.

2.2 Coordinate System

All internal positions use integer (x, z) grid coordinates. One grid unit corresponds to half a physical Erebus tile (the full tile is 2 grid units across). The starting position is stored in maze.start and all parity checks (coordInGrid) compare a coordinate's modulo-2 remainder against the starting coordinate's remainder to distinguish full-tile centres from half-tile edge midpoints. This distinction is critical for correct hole placement and tunnel detection.

The GPS returns values in metres. The conversion is: $\text{grid_coord} = \text{round}(\text{gps_metres} * 100 / 6)$. The factor of 100 converts to centimetres; dividing by 6 maps to the half-tile grid. Non-rounded floating-point GPS values (divisor=1) are used internally for dynamic turn angle calculations and victim deduplication proximity checks (within 10 GPS units).

3. Software Architecture

3.1 Main Control Loop

The controller runs at 32 ms per time step (31.25 Hz). Every time step the entire loop body is enclosed in a try/except safety net. The execution order within a single step is:

1. Update LiDAR point cloud from lidar.range_image slice [512*layer : 512*(layer+1)].
2. Read colour sensor; classify floor tile; update tunnel/checkpoint/swamp/hole markers.
3. Run victim detection pipeline on both cameras (skip if currently turning).
4. Detect obstructions via 8 distance sensors + 5 LiDAR angles.
5. If at a new grid node: call addCells(), updateToExplore(), chooseDirection().
6. If mid-turn: continue static_turn(); call addCells() and chooseDirection() on completion.
7. If obstruction_found: reverse to last_coords via dynamic_turn(), then replan.
8. Handle LoP signal from receiver.
9. Apply final motor velocities.

The robot spends most steps in state 7 (mid-turn) or state 9 (driving forward). States 5, 6, and 8 are triggered by discrete events. This structure ensures sensors are read and motors are set every single step with no blocking waits except inside delay() which is only called before victim scoring and at end-sequence.

3.2 Library Dependencies

Library	Use in This Project
NumPy	5x5 cell matrix storage, 2D occupancy grid for A*, neighbourhood matrix propagation, map serialisation and post-processing
OpenCV (cv2)	Camera image reading (imread), colour space conversion (BGR→HSV), pixel counting (inRange + countNonZero), ROI cropping for false-positive filters
PyTorch	CUDA device detection (torch.cuda.is_available()), GPU memory management
Ultralytics YOLO	YOLOv8 model loading and prediction; returns Results objects with boxes.cls, boxes.conf, boxes.xywh, boxes.xyxy
struct	Packing victim messages (struct.pack('i i c', x, z, type)) and map messages (struct.pack('2i', rows, cols)); parsing LoP signal
colorsys	Used in the custom bgra_to_hsv() conversion function for colour sensor classification
copy	Deep-copying the to_explore set and path sets to avoid shared-reference bugs during planning
math	Trigonometry for angle conversions, atan2 for dynamic turn targeting, sqrt for proximity checks

3.3 Path & Environment Auto-Detection

At startup the controller searches for the YOLO model file (best_erebus.pt) in a prioritised list: script directory, parent directories, then the current user's home/Desktop/Downloads folders. The active Windows username is obtained via `getpass.getuser()` so the path is always constructed correctly without any hard-coded username. A clear error message is printed if the model is not found.

A detected CUDA GPU is assigned as device 0 for all YOLO inference. If no GPU is found, inference falls back to CPU. This happens at module load time and requires no manual configuration per machine.

4. Map Representation

4.1 The Cell Class

Each explored location in the maze is represented by a Cell object containing a 5x5 integer matrix. The matrix is world-aligned: it is rotated in the Cell constructor based on `getRotation()` at the time of creation, so north is always up regardless of the robot's heading when the cell was first observed. This eliminates direction-dependent reading errors in all downstream code.

The 5x5 layout represents a physical half-tile in the Erebus world. The centre position (2,2) stores the tile type of the cell itself. The four cardinal edge positions (1,2)=north, (2,3)=east, (3,2)=south, (2,0)=west store direct wall readings. The four diagonal positions (0,0), (0,4), (4,0), (4,4) store corner wall inferences. The eight remaining off-centre positions in rows/columns 0,1,3,4 store distance sensor readings.

Tile type constants: 0=Empty, 1=Wall, 2=Hole, 3=Swamp, 4=Checkpoint, 5=Start. Tunnel tiles use values 14-20 (one value per colour, one additional for the second tile of each pair). Victim markers use values 30-36. Special internal markers: -1=Undefined, -2=TempWall, -3=StrongEmpty, -100=Void, 99=Room4 ().*

4.2 Cell Merging

When a new Cell is added at a coordinate already present in the map, the `merge()` method is called. It iterates over all 25 positions and takes `max(existing, new)`. The max rule means wall (1) always wins over empty (0), and victim markers (30+) always persist. A special exception prevents the merge from overwriting negative tunnel-colour values with positive wall values: if one cell has a negative value (tunnel) and the other has a value ≥ 30 (victim), neither overwrites the other.

4.3 The Net Class and Neighbour Propagation

Net is a sparse dictionary mapping (x,z) integer tuples to Cell objects. When any cell is added via `Net.__add__()`, the cell's 5x5 matrix is embedded at position [4:9, 4:9] of a 13x13 neighbourhood matrix (filled with -1). The code then iterates over the 24 surrounding cell coordinates (a 5x5 grid of coord offsets minus the centre) and extracts the corresponding 5x5 sub-section from the 13x13 matrix as a new Cell, merging it into the neighbour's existing cell.

This propagation mechanism ensures that a wall detected on the east side of cell A (stored at `A.cell[2][4]`) automatically appears on the west side of the adjacent cell B (stored at `B.cell[2][0]`) without any explicit synchronisation code. The same applies to all 24 neighbours, making the map globally consistent after every `addCells()` call.

When `addCells()` is called it also creates empty (unexplored) Cell stubs for all 8 immediate neighbours at distance 2. These stubs populate `to_explore` via `updateToExplore()` if they are reachable, ensuring the frontier always grows ahead of the robot.

4.4 Corner Wall Inference

After reading the 8 LiDAR directions (4 cardinal at R1 range, 4 diagonal at B1 range), `addCells()` applies geometric inference to fill in corner positions that the diagonal LiDAR scan may have missed. The rule is:

- If the front wall (`r_f == 1`) AND the right wall (`r_r == 1`) are both detected, the front-right corner must also be a wall (`b_fr = 1`), even if the 45° LiDAR ray returned 0.
- The same rule applies symmetrically to all four corners (front-left, back-right, back-left).

This prevents a common map artefact where the robot correctly maps two perpendicular walls but leaves a gap at their corner, causing A* to plan a diagonal path through a physically solid corner.

5. LiDAR Processing

5.1 Ray Selection and Noise Filtering

The LiDAR provides a complete 512-ray ring at the selected layer. The mapping from angle to array index is: $\text{index} = \text{round}(\text{angle} \% 360 * (512/360)) - 1 = \text{round}(\text{angle} * 1.422) - 1$.

`getLidarValue(angle)` reads 3 consecutive rays centred on the target angle and computes their mean and standard deviation. If the standard deviation is below 0.02 m (a smooth surface), the average is returned directly. If above 0.02 m (a sharp edge such as a corner), the function identifies the sub-pair (left or right) with the lower standard deviation and returns that pair's average. This prevents a wall corner from producing a spuriously large distance reading due to one ray hitting open space.

`getMinLidarValue(angle, fov)` is a secondary function that returns the minimum of all rays within a field-of-view window around the target angle, used for wide-angle obstructions.

5.2 Named Distance Ranges

`ckWallLidar(angle, limit)` checks whether the LiDAR reading at the given angle falls within a predefined distance range. 20 named ranges encode the expected distances from the robot centre to walls at different cell positions and orientations:

Code	Range (m)	Physical Meaning	Detection Behaviour
R1	0.040–0.065	Wall at half-tile distance (front/back/left/right)	Single-ray check; returns -1 if closer than min (obstacle very close)
R1.5	0.080–0.098	Wall at 1.5 half-tile distance	5-ray majority vote (>2 of 5 must be in range)
R2	0.103–0.125	Wall at full-tile distance (one tile away)	5-ray majority vote
R2.5	0.140–0.158	Wall 2.5 half-tiles away	5-ray majority vote
R3	0.163–0.185	Wall 3 half-tiles (1.5 tiles) away	5-ray majority vote

Code	Range (m)	Physical Meaning	Detection Behaviour
B1	0.063–0.090	Diagonal wall at half-tile corner distance	Single-ray; returns -1 if under-range
B2	0.145–0.173	Diagonal wall at full-tile corner	5-ray majority vote
B3	0.225–0.265	Diagonal wall at 1.5-tile corner	5-ray majority vote
K1	0.040–0.080	Near-corner asymmetric range (29° offset)	Used for ds0–ds7 angular distance sensors in Cell
CB	≤ 0.193	Corner-back diagonal threshold	Threshold-only (no upper limit)
CR	≤ 0.178	Corner-right diagonal threshold	Threshold-only (no upper limit)

6. Navigation System

6.1 Exploration Strategy

The robot maintains a `to_explore` set of grid coordinates that are known to exist (from LiDAR/neighbour propagation) but have not yet been physically visited. `updateToExplore()` is called every time the robot arrives at a new node. It removes the current coordinate from `to_explore`, then calls `getLegalMoves()` which checks all 8 octagonal neighbours for the `canBeAvailable` property (no wall at centre, no adjacent holes). Any legal neighbour that is not yet in `to_explore` and not yet explored is added. Holes and cells adjacent to tunnels are permanently excluded.

The robot continues exploring until `to_explore` is empty. At that point `chooseDirection()` enters its tunnel-fallback recursion, which sequentially tries to re-add each known tunnel coordinate to `to_explore` and re-run path planning. If all tunnels have been visited and the start cell is reachable, the robot navigates back to start and calls `initiateEndSequence()`. If no path to start exists, the end sequence is triggered immediately.

6.2 A* Path Planning (`getBestPath`)

`getBestPath()` builds a dense 2D NumPy occupancy array from the sparse Net dictionary. The bounding box (`getCorners()`) determines the array dimensions. Wall cells are set to 1, all others to 0. Cells that are holes or near tunnels are also excluded from traversal.

The core function `pathAStar(curr_map, offset, origin, destination)` runs a modified Dijkstra/A* search. The distance function between adjacent cells adds a turn-penalty of 0.5 when the required heading differs from the robot's current heading. This makes the planner prefer straight-line paths over zigzag paths of equal Manhattan length.

After running A* to every cell in `to_explore`, `getBestPath()` selects the destination with the minimum total cost. An additional small penalty of -0.5 is applied to destinations with neighbours already marked as walls (cells deeper into dead-ends are slightly preferred over cells at junctions, helping the robot clear dead-ends before returning to explore branches).

6.3 `chooseDirection` and Tunnel Fallback

`chooseDirection()` takes the current path (`curr_path`, a set of coordinates) and intersects it with `getLegalCoordAngle()` — a dictionary of `{neighbour_coord: absolute_angle}` for all reachable adjacent cells. The next coordinate in the path that appears in the legal-moves dictionary is selected; `moveInDirection()` sets the `turn_multi` and `next_angle` for the corresponding heading.

If `curr_path` is `None` or empty and `to_explore` is also empty, `findBestPath()` enters a recursive tunnel fallback. It tries each coloured tunnel object in the order BLUE → BLUE2 → PURPLE → PURPLE2 → RED → RED2 → GREEN → GREEN2 → YELLOW → YELLOW2 → ORANGE → ORANGE2, adding the tunnel's coordinate and its unexplored neighbours to `to_explore` and retrying A*. A maximum recursion depth of 3 is enforced; beyond that, the robot submits the current map and exits cleanly.

A special case handles the situation where the robot is physically standing on a tunnel tile with an empty path (A* returns {}). In this case the robot immediately moves toward the nearest unexplored neighbour without incrementing the recursion counter, triggering the tunnel crossing and room change via the colour sensor on the following step.

6.4 Static Turn

In-place turns are executed by `static_turn()`, which is called every step while `next_angle != -1`. The turn speed profile has three zones based on angular distance to target:

- Distance > 6°: full speed (`turn_multi = ±1`, `speeds = ±MAX_VELOCITY`).
- Distance 0.2°–6°: reduced speed proportional to $\sqrt{\text{angular_distance} / 90}$. This gives a smooth deceleration curve as the robot approaches the target heading.
- Distance < 0.2°: micro-speed equal to $\text{angular_distance} / 10 * \text{MAX_VELOCITY} + 0.005$. This prevents overshoot at the final degrees.
- Distance < 0.001°: turn complete. `turn_multi` resets to [1,1]; `chooseDirection(next_move)` is called to immediately begin the next movement.

The function also handles the 0°/360° wraparound problem: when the robot is pointing near north, both `curr_angle` and `next_angle` are shifted by 180° before comparison to avoid spurious sign flips that would cause the robot to spin the wrong way.

6.5 Dynamic Turn (Obstruction Recovery)

When `obstruction_found` is `True`, `dynamic_turn(objective_gps, current_gps)` is called to steer the robot back to `last_coords`. It computes the angle from current GPS position to target GPS position using `atan2`, rounds to the nearest 45° octagonal heading, and applies an exponential scale factor:

$\text{scale_factor} = 0.33 + e^{-0.1 \cdot d} - 0.33 \cdot e^{-0.3 \cdot d}$ where $d = \text{angular_distance} / 180$.

This curve gives approximately full speed at large angles and tapers to ~0.33 at zero, preventing oscillation around the target. The asymmetric application (scale applied to only one wheel) produces the curving reversal needed to avoid obstacles during recovery.

7. Floor Tile and Tunnel Classification

7.1 Colour Sensor Processing

At each step, `getColor()` reads one BGRA pixel from the colour sensor, unpacks it with `struct.unpack('BBBB', ...)`, and converts to HSV via a custom `bgra_to_hsv()` function using the `colorsys` algorithm (all channels normalised to 0–1 then scaled). The resulting (H, S, V) tuple is then passed to `checkColor()` which performs a simple range check against the tile's HSV min/max dictionary.

7.2 Tunnel Tile Objects

Each colour tunnel is represented by a Tunnel object with fields: `val` (integer tile type 14-20 for primary, 15-20 for secondary), `color_range` (HSV range dict), `found` (bool), and `coord` (grid coordinate once

detected). Each colour has two Tunnel instances (e.g. `_BLUE` and `_BLUE2`) because each physical tunnel consists of two tiles: an entry tile and an exit tile on the other side.

`checkAndPlaceTunnel()` is called each step when the floor colour matches a tunnel range. It checks that: (1) this tunnel's first tile has already been found (for the secondary-tile objects), (2) the current forward coordinate is on the integer grid (`coordInGrid`), and (3) the coordinate is not already registered in the `_TUNNELS` dictionary. On a successful match it marks the tile as explored, adds it to `_TUNNELS` with the colour name, and proactively adds the tunnel coordinate and all its unexplored cardinal neighbours to `to_explore`.

The `obstruction_found` flag is explicitly cleared when a tunnel is detected, because the dark or coloured tunnel tile can superficially resemble an obstacle to the distance sensors.

7.3 Room Transition Logic

`_TUNNELS` is a dictionary mapping tunnel-tile grid coordinates to their colour names. When the robot arrives at a new node and `last_coords` (the previous position) is in `_TUNNELS`, `changeCurrentRoom()` is called with the colour. The lookup table for normal transitions is:

Tunnel Colour	From Room	To Room	Connects
BLUE	1	2	Rooms 1 ↔ 2
YELLOW	1	3	Rooms 1 ↔ 3
GREEN	1	4	Rooms 1 ↔ 4 (elevated area)
PURPLE	2	3	Rooms 2 ↔ 3
ORANGE	2	4	Rooms 2 ↔ 4 (elevated area)
RED	3	4	Rooms 3 ↔ 4 (elevated area)

If the robot's tracked room does not match either endpoint of the tunnel colour (tracking error), the fallback selects the endpoint room that the robot is currently NOT in according to the lookup. This recovers from single missed tunnel crossings without manual intervention.

Room assignment is stored per-cell at `cell.room` and is preserved during Cell merges via explicit `old_room` restoration. All cells in a given physical room receive the same room value, which is later used to determine Room 4 boundaries for map serialisation.

8. Hole and Swamp Detection

8.1 Hole Detection Robustness

Holes are detected when the depth sensor (a downward-facing distance sensor) or the colour sensor returns a reading consistent with a black void. Before writing a hole to the map, `placeAndHandleHole()` performs a bounding-box guard: it computes the axis-aligned bounding box of all currently mapped non-hole cells. If the candidate hole coordinate falls outside this bounding box, it is rejected and only an obstruction flag is set. This prevents phantom holes from being placed at the edge of the known world, where the camera or depth sensor may be sensing open space beyond the maze boundary rather than a real hole.

The hole placement function also applies a `coordInGrid()` snap: if the candidate coordinate does not lie on the expected tile-centre parity, it is shifted by one grid unit in the front direction (and if still off-parity, shifted again by 270°) to snap to the nearest valid tile centre. This handles sub-tile GPS offset that can occur when the robot is mid-cell.

8.2 Swamp

When the colour sensor detects a swamp tile, the cell is marked with tile type 3 (`_SWAMP`). The `insideSwamp()` function checks whether the current cell contains any swamp value in its 5x5 matrix. If true, the planned forward speed is reduced by 32%, accurately modelling the Erebus swamp movement penalty.

9. Map Serialisation

9.1 Overview

When exploration is complete, `getMap(nice_view=False)` is called to produce the numpy array that will be transmitted to the supervisor. The function applies nine post-processing stages to transform the internal sparse-dictionary map into a clean, competition-ready output:

10. Layout: All cells in the sparse dictionary are arranged into a (width × height) grid of 5x5 sub-matrices based on their coordinates and the computed bounding box.
11. Stitching: The 5x5 sub-matrices are merged vertically then horizontally. Overlapping edge rows/columns are combined with `max()`, so shared wall edges between adjacent cells consolidate correctly. `shiftMatrix()` (a direction-parameterised list-rotation utility) handles the iterative merge.
12. Margin trim: A 2-cell margin is removed from all sides to eliminate boundary artefacts.
13. Marker cleanup: `TempWall` and `StrongEmpty` markers are both converted to 0 (empty).
14. Node-centre undefined → empty: All positions at odd row, odd column (node centres that were never visited) are set to 0. This is correct because the robot never physically traversed those positions, but they are passable space.
15. Wall/corner undefined → wall: All remaining `_UNDEFINED` positions (which are at wall/corner slots the robot never scanned) are set to 1. This implements the competition rule that unobserved boundaries default to wall.
16. Phantom wall removal in hazard clusters: For any wall marker that has the same hazard tile type (hole, swamp, checkpoint, start) on both sides in the same row or column, the wall is changed to empty. An iterative flood-fill pass then cleans up any surrounded wall markers inside an all-tile neighbourhood. This handles the case where two adjacent same-type tiles would have a spurious wall inserted between them by step 6.
17. BFS connectivity filter: Starting from the centres of all explored cells, a breadth-first search spreads through all non-wall positions. Any filled (non-empty) position that is not reachable from this BFS seed AND has no explored neighbour within 8-connectivity is converted to empty. This removes LiDAR artefacts that appear as phantom wall clusters outside the actual maze boundary.
18. Room 4 fill: All cells with `room == 4` are identified. Their tight bounding box is computed. The box is extended to the nearest map edge on whichever sides are closer to the maze boundary. Every position inside the extended box is set to the Room 4 marker (*). Explored cells from rooms 1–3 and all tunnel cell footprints within the box are then restored to their original values.

9.2 Transmission Format

After `getMap()` returns the numpy array, `initiateEndSequence()` performs a pre-submission cleanup: all `TempWall` markers still present in the raw cell dictionary (not yet overwritten by real sensor data) are converted to `StrongEmpty`, preventing them from appearing as real walls in the output. Then:

- The array shape (rows, cols) is packed as `struct.pack('2i', ...)` — two 32-bit integers.
- The array is flattened to a 1D list and joined with commas, then UTF-8 encoded.
- Shape bytes and matrix bytes are concatenated and sent via `emitter.send()`.
- A map-evaluate request byte ('M') is sent to request the map multiplier score.
- After a 5-second delay, an exit byte ('E') is sent to claim the exit bonus.

10. Victim and Hazard Detection

10.1 Detection Architecture

Both cameras produce a 64x32 pixel image each step. The detection pipeline runs on both cameras independently and in parallel (sequential calls). A six-stage filter cascade validates each YOLO detection before the victim is recorded and reported.

Victim Category	Detection Chain
H, S, U (survivor symbols)	Pre-filter → YOLO predict → area+conf+ratio check → black-on-white ROI check → centerness check → deduplication
C, F, O, P (hazmat signs)	Pre-filter → YOLO predict → area+conf+ratio check → HSV ROI colour check → centerness check → deduplication

10.2 Stage 1 — Pre-Filter (Wall-Only Image Rejection)

`is_wall_only_image()` converts the raw BGR frame to HSV and counts two pixel populations:

- Dark wall: $V < 50$ across the full 0–180 hue range (almost black).
- Cyan wall: H in $[80, 110]$, $S > 55$, V in $[50, 225]$ (the teal-coloured Erebus walls).

If either population exceeds 90% of total pixels, the frame is immediately discarded without calling YOLO. In practice this filter eliminates most empty wall frames. It runs in ~0.1 ms vs. ~5–20 ms for a YOLO inference, so the CPU savings are significant over an entire maze run.

10.3 Stage 2 — YOLO Inference

The YOLOv8 model has 7 output classes: 0=C, 1=F, 2=H, 3=O, 4=P, 5=S, 6=U. A temporary image is written to disk and `predict()` is called with `verbose=False` to suppress per-frame output. Only the top detection (highest confidence) is evaluated in the subsequent filter stages, though all detections are logged with timestamp, class, confidence, area, and aspect ratio for debugging.

10.4 Stages 3–5 — Geometric and Pixel Filters

Filter	Applies to	Thresholds	Rationale
Bounding-box area	All	$80 < \text{area} < 1300$ (H/S/U) $80 < \text{area} < 1400$ (hazmats)	Too small = noise; too large = full-frame wall artefact (cyan walls produce area $> 1200 \text{ px}^2$).

Filter	Applies to	Thresholds	Rationale
Confidence threshold	All	conf > 0.55 (H/S/U) conf > 0.65 (hazmats)	Hazmats require higher confidence because their shapes are more ambiguous against textured walls. The 'U' (trident) class uses the lower threshold as it rarely scores above 0.65.
Aspect ratio w/h	All	0.2–2.5 (H/S/U) 0.5–2.0 (hazmats)	Hazmats are roughly square (ratio ≈ 1.0). An elongated bounding box (ratio < 0.5) most likely corresponds to a wall edge or a thin shadow, not a sign.
Black-on-white ROI	H, S, U only	white fraction ≥ 0.10 in bounding box ROI	Crops the bounding box from the raw image and counts bright pixels ($V > 180$). Real victims have a white rectangular background with a dark symbol. Cyan walls have almost no white inside any detected ROI (measured white < 0.05).
HSV ROI colour check	C, F, O, P only	dark + cyan fraction ≤ 0.50 in ROI	Sums the dark ($V < 55$) and cyan ($H 80\text{--}110$) pixel fractions inside the bounding box. Real hazmat signs have coloured backgrounds (orange, red, yellow, green). A black cross or dark shadow on a cyan wall will fail this check.
Centerness	All	image centre-x within 20–80% of frame width	A detection near the extreme edge of the frame means the robot is far to one side of the victim and the GPS coordinate does not correspond to the wall the victim is actually on.

10.5 Spatial Deduplication (addToSeen / prev_victims)

To prevent the same physical victim from being scored multiple times (e.g. as the robot passes by at different speeds), `addToSeen()` inserts 12 tuples into the `prev_victims` set. Each tuple encodes (`coord_x`, `coord_z`, `rotation_index`, `camera_side`). The 12 tuples cover:

- Current coordinate + rotation $\pm 0, \pm 1$ (both cameras): 6 tuples.
- The forward-adjacent coordinate (`front_coord`) + rotation $\pm 0, \pm 1$ (both cameras): 6 tuples.

Before any detection is accepted, the detection tuple is checked against `prev_victims`. This approach blocks re-detection from the same cell and adjacent cells, from both cameras, and at slightly different headings ($\pm 45^\circ$), making it robust to the robot's real-world position uncertainty.

An additional geometric deduplication layer operates in `scoreVictim()` using the `victim_pos` set: before sending a victim message, the robot checks all previously sent victims and rejects any new detection whose angular direction overlaps an already-sent victim's angular window (based on camera FoV) AND whose GPS distance is less than 10 GPS units. This catches any victims that slipped past the `prev_victims` set due to non-standard robot trajectories.

10.6 Victim Scoring Protocol

When a detection passes all filters:

19. The robot stops immediately (motor velocities = 0).
20. `delay(1400 ms)` is called to satisfy the minimum stopping time required by the Erebus scoring rules.
21. A victim message is packed as `struct.pack('i i c', gps_x, gps_z, victim_type_char)` and sent via emitter.

22. The victim coordinates and type are written to the Cell matrix at the correct wall position (north/east/south/west face) based on camera side and robot heading.

10.7 Victim Position in the Map

`_placeVictimInMap()` writes the victim marker to the correct position in the Cell matrix only when `grid_position == 'middle'` (the robot is precisely aligned laterally with the victim). 'front' and 'back' detections are scored for points but not written to the map, preventing victims from appearing on the wrong cell's wall face.

The wall direction is computed as: `wall_dir = (robot_heading + 90°) % 360` for the right camera, and `(robot_heading - 90°) % 360` for the left camera. This is then mapped to the four cardinal positions in the world-aligned Cell matrix: north=(0,2), east=(2,4), south=(4,2), west=(2,0).

11. YOLO Model Training

11.1 Dataset Collection

Training images were collected by running the robot through multiple manually designed Erebus mazes with known victim and hazmat placements. Screenshots were captured from both cameras at varying distances (corresponding to area values from ~80 to ~1300 px²) and both major wall colour environments (dark and cyan). Each image was labeled with the YOLO bounding-box format using a labeling tool, producing annotations for all 7 classes.

11.2 Data Augmentation

To improve generalisation across the variable lighting and wall colours in Erebus, the training set was augmented with: horizontal flips (all classes are symmetric), brightness and contrast jitter ($\pm 20\%$), small rotations ($\pm 5^\circ$), and minor scaling. Augmentation was applied online during training using the Ultralytics built-in pipeline.

11.3 Training Configuration

Training used Ultralytics YOLOv8 (ultralytics library) with the standard yolov8n (nano) or yolov8s (small) base architecture, fine-tuned on the Erebus dataset. The best checkpoint by validation mAP50 was saved as `best_erebus.pt`. The 7 output class indices were then remapped in the controller to the character labels: 0=C, 1=F, 2=H, 3=O, 4=P, 5=S, 6=U.

11.4 Known Model Limitations

- The 'U' (trident) class achieves lower confidence scores than other classes due to its complex three-pronged shape. The confidence threshold was lowered to 0.55 and the area upper limit raised to 1300 to capture close-range trident detections.
- All three survivor symbols (H, S, U) can be confused with black structural features on cyan walls when seen at extreme angles. The black-on-white ROI filter was specifically designed to resolve this, as it operates at the pixel level rather than relying on the neural network alone.

12. Lack of Progress and Error Handling

12.1 LoP Signal Handler

The Erebus supervisor sends the byte 'L' to the receiver when it detects a lack of progress. The controller checks `receiver.getQueueLength()` every step and immediately handles the packet:

23. `turn_multi` is reset to [1,1] (forward state) and speeds are zeroed.
24. `obstruction_found` is cleared.
25. Both `last_coords` and `last_obstacle_coord` are re-added to `to_explore` (so the robot will attempt to revisit the position it was stuck at rather than treating it as permanently blocked).
26. `getBestPath()` is called to plan a new route from current position.
27. `chooseDirection()` is called. If it raises any exception, `static_turn()` is invoked as a fallback, which physically rotates the robot toward the nearest legal move.

12.2 Universal Exception Safety Net

The entire main loop body (all ~400 lines of sensor reads, detection, map updates, and navigation) is wrapped in a single `try/except` block:

- On any exception: the error type, message, robot position, and current room are logged with `traceback`. `curr_path` is set to `None`. `obstruction_found` is cleared. The last safe motor speed is re-applied.
- A global counter `_LOOP_ERR_COUNT` tracks consecutive failures. If it reaches 40 without a clean step in between, `initiateEndSequence()` is called immediately to submit whatever map has been built, preserving partial score.
- If an exception occurs during `initiateEndSequence()` itself, it is caught by an inner `try/except` and printed but does not prevent the `exit()` call that follows.

The safety net is designed for unexpected Python exceptions (`AttributeError`, `IndexError`, etc.) caused by edge cases in maze geometry or sensor readings. It is not a substitute for correct navigation logic; it is a last resort that ensures the robot always submits something rather than crashing silently and scoring zero.

12.3 initiateEndSequence Guard

A module-level boolean `_END_SEQUENCE_DONE` prevents `initiateEndSequence()` from being called more than once per run, even if the safety net triggers it multiple times in rapid succession. Before transmitting the map, all `TempWall` markers in the raw cell dictionary are reset to `StrongEmpty` (which the serialiser converts to empty), ensuring no sensor-noise artefacts appear in the submitted map.

13. Performance Evaluation

13.1 Testing Methodology

All development was validated using the Erebus built-in random maze generator. Test runs spanned: small/medium/large maze sizes; 0 to maximum victim density; all four room configurations; mazes with and without all six tunnel colours; mazes with multiple holes and swamps. Hundreds of runs were executed across the development cycle with console logs captured for post-analysis.

13.2 Key Performance Metrics

Metric	Observed Results
Full maze exploration completion	High across standard maze sizes. The robot completes exploration and calls <code>initiateEndSequence()</code> without manual restart in the large majority of runs.
False positive victim rate	Very low after the six-stage filter was deployed. Pre-filter alone rejects roughly 40–60% of frames before YOLO is called. The ROI colour and black-on-white filters eliminate most remaining false positives from wall artefacts.
Map wall accuracy (rooms 1–3)	High. Corner inference and BFS phantom removal significantly improved accuracy compared to raw LiDAR output. Undefined wall positions correctly default to wall.
Room 4 (*) fill correctness	Correct in all tested configurations where at least one Room 4 cell is reached. The tight bounding-box algorithm with edge extension handles all tested room-4 corner positions.
Victim deduplication	No duplicate reports observed in standard maze configurations. The 12-tuple <code>prev_victims</code> system combined with the <code>victim_pos</code> angular check provides two independent layers of protection.
LoP recovery	The robot successfully resumes exploration after LoP in most cases. The <code>static_turn</code> fallback handles the rare case where the path planner cannot immediately find a new route.
Exception rate	The 40-error threshold in the universal safety net has never been reached in any non-buggy build. The safety net fires 0–2 times per run in edge-case maze geometries.

13.3 Known Limitations and Future Work

- Diagonal movement at corridor corners: when the robot moves diagonally (45° headings), it occasionally clips walls at concave corners. Improved pre-turn clearance checks using the K1/K2 LiDAR ranges would reduce this.
- YOLO 'U' class accuracy: the trident symbol still occasionally misclassifies at extreme viewing angles. Collecting more training data from oblique angles and partially occluded views would improve this.
- A* rebuild cost: `getBestPath()` rebuilds the full occupancy array from the sparse dictionary on every call. For large mazes this can take several milliseconds. Incremental dirty-cell updates would reduce per-step latency.
- Room 4 tracking robustness: a single missed tunnel crossing can offset the room counter, causing cells to receive incorrect room assignments and potentially mis-filling Room 4. A sensor-fusion approach combining floor colour, GPS position, and room transition history would make this more robust.

14. Lessons Learned

Participating in both RoboCup Croatia 2026 and continuing development for the World Championship has produced a number of hard-won insights:

- **Context beats confidence:** Most false-positive detections were not caused by a weak YOLO model but by applying the model's output without context verification. A prediction with 0.75 confidence on a wall texture is still wrong. The multi-stage filter (especially the pixel-level ROI checks) exists entirely to provide context the neural network does not have.
- **Map post-processing is non-trivial:** A raw occupancy grid from LiDAR data contains phantom walls (artefacts behind outer boundaries), undefined positions, and intra-tile gaps. Each of the nine serialisation stages was added to fix a specific class of submission failure discovered during testing.
- **Defensive programming pays off in competitions:** The universal exception safety net has never saved a run from a zero score, but knowing it exists lets the team add new features without worrying that a rare edge case in one function will crash the entire run.
- **Half-tile coordinate system:** Using grid units equal to half a physical tile ($\text{GPS} * 100 / 6$) gives the robot the ability to represent the midpoints between tile centres as distinct grid coordinates. This allows wall edges and diagonal corners to be stored without sub-dividing the Cell matrix further, keeping the data structure compact.
- **Remote teamwork requires discipline:** With team members in different cities, maintaining a shared codebase required a clear naming convention, comment standards, and test protocols. The console log system (with clear prefixes like [VICTIM], [LOP], [ROOM], [YOLO]) was essential for debugging issues reported by a team member who was not present when the run was executed.

15. Conclusion

The UNIQA CRO Team RoboCraft has developed a fully autonomous Rescue Simulation robot that combines LiDAR-based grid mapping with a custom A* planner, six-stage false-positive-suppressed YOLO victim detection, colour-sensor-driven multi-room tracking, and a nine-stage map serialisation pipeline. The system is the result of sustained iterative development from the Robotics Summer School in Hvar through the Croatian national qualifier and into the World Championship preparation.

The key technical contributions of this project are: (1) the 5x5 Cell matrix with world-aligned rotation and full-neighbourhood propagation that makes the occupancy grid globally consistent; (2) the pixel-level ROI filter cascade that resolves the false-positive challenge specific to Erebus's wall colour palette; and (3) the Room 4 tight-bounding-box algorithm that correctly marks the elevated area in all tested maze configurations.

The team looks forward to the World Championship as an opportunity to benchmark the system, receive feedback from international judges, and continue developing skills in autonomous robotics and applied AI.

References

- RoboCupJunior Rescue Simulation 2026 Rules — <https://junior.robocup.org/>
- Erebus simulation platform documentation and tutorials — <https://erebus.rcj.cloud/docs/tutorials/>
- Webots Sensor API Reference — <https://cyberbotics.com/doc/guide/sensors>

- Ultralytics YOLOv8 documentation — <https://docs.ultralytics.com/>
- OpenCV Python documentation (inRange, cvtColor, countNonZero) — <https://docs.opencv.org/>
- NumPy documentation (array operations, BFS implementation) — <https://numpy.org/doc/>
- PyTorch CUDA documentation — <https://pytorch.org/docs/stable/cuda.html>
- RoboCupJunior forum — Rescue Simulation category — <https://junior.forum.robocup.org/>
- Python 3 standard library (struct, colorsys, math, copy) — <https://docs.python.org/3/>